

IA Aplicada com LLMs

Aula 05 — Confiabilidade

O estado mínimo que este deck exige

```
@dataclass
class Estado:
    objetivo: str          # a âncora contra deriva
    passos: list[Passo]    # a trajetória – permite detectar laço
    tokens_gastos: int      # e o orçamento
    termino: Termino | None # POR QUE parou
    motivo: str | None     # qual teto, qual erro
    historico: list[dict]   # o que o modelo vê – DERIVADO
```

mensagens[] é transporte, e não o estado do agente. Seis campos, e nenhuma salvaguarda deste deck existe sem eles.

O objeto **completo** é da Aula 08 — aqui basta saber o que cada campo sustenta.

Os oito modos de falha da Aula 01, virando código

Falha	Onde resolvemos
Laço	§6
Deriva de objetivo	§7
Alucinação de argumento	§3, §4
Erro em cascata	§3, §8
Custo descontrolado	§1
Silêncio	§2, §10
Ferramenta errada	deck anterior
Contexto estourado	próximo deck

O aviso que abre o deck: nenhuma salvaguarda é grátis

- Orçamento apertado **mata tarefa legítima**
- Detector agressivo **interrompe quem progredia devagar**
- Confirmação em excesso **mata a autonomia**

Escolher a dose é engenharia, não receita.

Parte 1

Orçamento e terminação

max_passos=6 não é orçamento

Passo não é unidade de custo. Consultar um id: ~300 tokens.

Ler um documento de 30 páginas: ~40.000. "No máximo 10 passos"

= "no máximo entre 3 mil e 400 mil tokens". Isso não é um limite.

```
@dataclass
class Orcamento:
    max_passos: int = 12
    max_tokens: int = 60_000
    max_reais: float = 0.50
    max_segundos: float = 120.0

    def excedido(self, estado) -> str | None:
        """Devolve QUAL teto estourou – o motivo importa."""
```

Quatro observações sobre esse código

É parâmetro, não constante. Triagem e investigação não têm o mesmo direito de gastar.

Unidade monetária vem da Aula 02 — é a moeda compreendida pela gestão.

Tempo de parede é o teto esquecido — pega a ferramenta travada, que não estoura nenhum dos outros três.

Devolve *qual* teto estourou — `bool` seria simples e inútil: morrer por tempo e morrer por tokens têm diagnósticos opostos.

As quatro formas de terminar

1. RESPONDEU	devolveu sem <code>tool_calls</code>	sucesso
2. ORCAMENTO	um dos quatro tetos excedido	inconcluso
3. ERRO_FATAL	não há como prosseguir	falha
4. HUMANO	execução suspensa aguardando confirmação	suspenso

Na Aula 03: a 1ª era `return`, a 2ª era `raise`, a 3ª derrubava o processo e a 4ª não existia. Agora as quatro são **dado**.

```
finally:  
    registrar_trace(estado)           # sempre, inclusive nas falhas
```

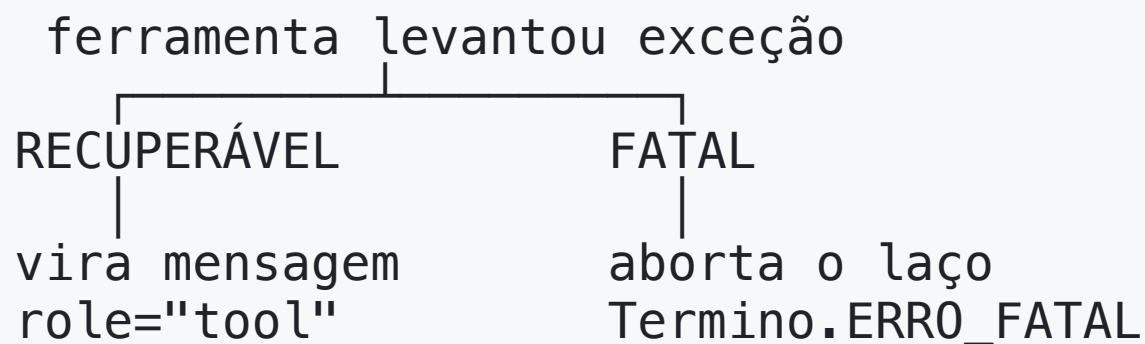
Um tratamento que registre apenas o sucesso assegura que a causa jamais será identificada.

A linha "**silêncio**" da tabela de falhas morre aqui.

Parte 2

Erro

A decisão mais importante do deck



Recuperável — o modelo contorna mudando a chamada: argumento inválido, registro inexistente, formato errado.

Fatal — nada que o modelo decida resolve: credencial, banco fora, disco cheio. Devolver um fatal ao modelo = quatro reformulações educadas até o orçamento acabar.

A classificação cabe ao código. Não ao modelo.

A pergunta que classifica

Existe alguma chamada diferente que o modelo poderia fazer para contornar?

Se existe → **recuperável**, e o erro deve dizer qual é.

Na dúvida: origem no **argumento** = recuperável.

Origem na **infraestrutura** = fatal.

A mensagem de erro também é prompt

Aula 03: *"a descrição da ferramenta é prompt"*. O corolário que ninguém enuncia: o erro é o único texto que o modelo lê para decidir **como se corrigir**.

```
{"erro": "falhou"} # inútil

{"erro": "ValueError: invalid literal for int()..."} # verdadeiro,
# e sem ação possível

{"erro": "formato de id inválido", # útil
 "esperado": "D seguido de 4 dígitos, ex: D-4471",
 "recebido": "4471",
 "sugestao": "chame listar_despesas"}
```

O terceiro responde: **o que errou · qual era o certo · o que fazer agora**.

Retry: onde vale e onde não

Falha	Quem resolve	Como
429 , 500 , timeout	o código	backoff exponencial
argumento inválido	o modelo	erro volta como observação
credencial, serviço fora	ninguém	ErroFatal

```
for tentativa in range(3):                # 0 ANTI-PADRÃO
    try:
        return FERRAMENTAS[nome](**argumentos)
    except ErroRecuperavel:
        continue        # mesmos argumentos. Mesmo erro.
```

Retry automático é para **transporte**. Conteúdo volta para o modelo.

Parte 3

Laço, deriva, escrita

max_passos limita o dano. Não detecta o laço.

Com teto de 12, um agente preso na 1ª volta gasta 12 passos e devolve o diagnóstico **errado**: "orçamento esgotado".

```
def assinatura(passo) -> tuple:  
    return (passo.ferramenta, json.dumps(passo.argumentos, sort_keys=True))
```

O `sort_keys=True` não é detalhe: sem ele, o detector não detecta nada.

Progresso nulo é a versão sutil — as chamadas *variam*, nada avança:

```
politica → historico → politica → despesa → politica → historico
```

Cada chamada difere da anterior. O **conjunto** se repete.

O que fazer ao detectar

1. **Injetar observação** — *"a ferramenta já foi chamada com estes argumentos e devolveu este resultado"*
(mais barata, e costuma bastar)
2. **Reduzir as ferramentas ativas** — força outro caminho
3. **Abortar** com motivo `laco_detectado` — diagnóstico honesto

Preço: detector agressivo dispara falso positivo em quem legitimamente varre uma lista. Comece frouxo.

Deriva de objetivo

Passo 30: o pedido original está cercado de 12.000 tokens.
É exatamente o **meio** do contexto (*lost in the middle*).

```
if estado.n_passos % REANCORAR_A_CADA == 0:  
    msgs.append({"role": "user",  
                "content": f"Lembrete do objetivo: {estado.objetivo}"})
```

Dezenas de tokens a cada 5 passos.

A melhor relação custo-benefício da aula.

Idempotência: agora não é mais hipótese

As salvaguardas que acabamos de adicionar **aumentam** a chance de repetir uma escrita: o **retry** reenvia o que já teve efeito, e a **detecção de laço** deixa o agente continuar e repetir.

```
if (existente := PARECERES.get(chave)):  
    return {**existente, "ja_existia": True}
```

`uuid4()` a cada chamada **não** é chave de idempotência — é um identificador novo por tentativa. E o `ja_existia: True` é informação **para o modelo**: ele descobre que já agiu.

O exemplo que fecha o deck

SEM detector, erro inútil:

```
passos 0..11  consultar_historico {"funcionario": "F-88"} -> erro  
TERMINO: orcamento_esgotado | 14.200 tokens | R$ 0,036
```

COM erro que ensina:

```
passo 0  -> {"erro": "...", "esperado": "F-088",  
            "sugestao": "chame listar_funcionarios"}  
passo 1  consultar_historico {"funcionario": "F-088"} -> ok  
TERMINO: respondeu | 1.480 tokens | R$ 0,004
```

O detector **nem disparou**. Quem resolveu foi o texto do erro.

O subproduto que vale mais que o produto

Para detectar laço → guardamos ferramenta e argumentos.

Para o orçamento → tokens e custo. Para o término → o motivo.

Isso é um trace, matéria-prima da aula de **evals**, e da **observabilidade** em produção.

■ Não se avalia o que não se rastreia.